Signed, Sealed, Supersingular

Post-quantum signatures and the Supersingular Isogeny Path Problem

Owen Waldron

UW PMC S26 SASMS 2

July 15, 2026

Outline and Goals

1. What a digital signature even is
2. An important hard problem: the Supersingular Isogeny Path Problem
3. Why we believe that problem is hard
4. How you build a signature on top of it

Goal: a taste for the fun math underpinning post-quantum crypto!

Section 1

What is a digital signature?

The Digital Signature algorithms

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ Generate a public/private key pair

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages
- **Verify**$(pk, m, \sigma) \to \{0, 1\}$ — anyone can verify a signature using the public key

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages
- **Verify**$(pk, m, \sigma) \to \{0, 1\}$ — anyone can verify a signature using the public key

This is an essential primitive for verifying the source of messages. For example, when you visit google.ca, you want to verify that the HTML response you get was in fact delivered by Google.

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages
- **Verify**$(pk, m, \sigma) \to \{0, 1\}$ — anyone can verify a signature using the public key

This is an essential primitive for verifying the source of messages. For example, when you visit google.ca, you want to verify that the HTML response you get was in fact delivered by Google.

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages
- **Verify**$(pk, m, \sigma) \to \{0, 1\}$ — anyone can verify a signature using the public key

This is an essential primitive for verifying the source of messages. For example, when you visit google.ca, you want to verify that the HTML response you get was in fact delivered by Google.

- **Forge**$(pk, m) \to \sigma$ — an antagonist, *without* the secret key

The Digital Signature algorithms

- **KeyGen**$() \to sk, pk$ — Generate a public/private key pair
- **Sign**$(sk, m) \to \sigma$ — use private key to sign messages
- **Verify**$(pk, m, \sigma) \to \{0, 1\}$ — anyone can verify a signature using the public key

This is an essential primitive for verifying the source of messages. For example, when you visit google.ca, you want to verify that the HTML response you get was in fact delivered by Google.

- **Forge**$(pk, m) \to \sigma$ — an antagonist, *without* the secret key

Signing and verifying are **easy** (fast). Forging a signature for *any message* must be **hard** (slow).

How do we trust a Digital Signature scheme?

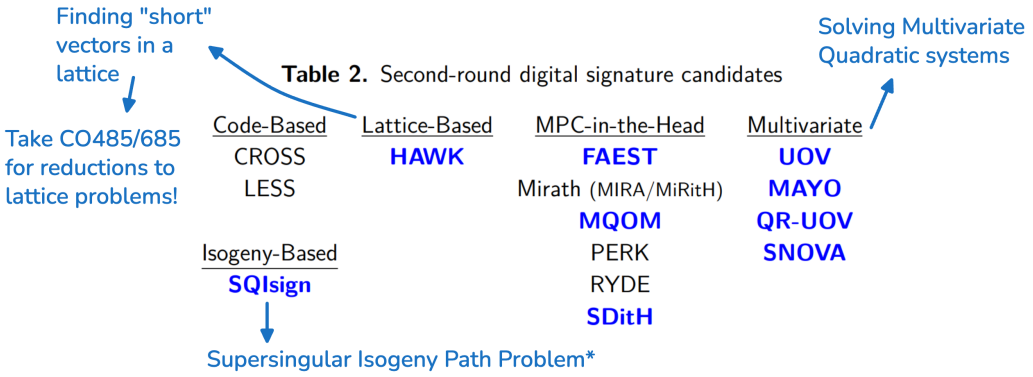
We can show that **forging reduces to solving a hard problem** with polynomial-time reductions.
Break the signature $\Rightarrow$ you broke the problem.

Table 2. Second-round digital signature candidates

Code-Based	Lattice-Based	MPC-in-the-Head	Multivariate
CROSS	**HAWK**	**FAEST**	**UOV**
LESS		Mirath (MIRA/MiRitH)	**MAYO**
		MQOM	**QR-UOV**
Isogeny-Based		PERK	**SNOVA**
SQIsign		RYDE	
		SDitH	

How do we trust a Digital Signature scheme?

We can show that **forging reduces to solving a hard problem** with polynomial-time reductions.
Break the signature $\Rightarrow$ you broke the problem.



Section 2

Curves, isogenies, and a giant graph

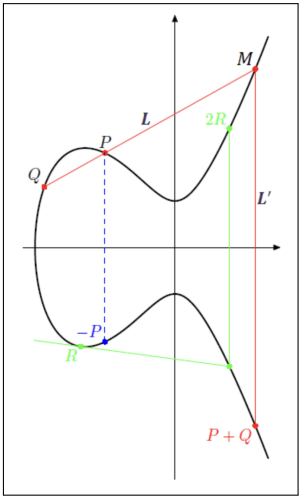
Elliptic curves: our vertex building blocks

Over a field, an elliptic curve is the set of points (x, y) that satisfy

$$y^2 = x^3 + ax + b.$$

along with a sentinel "point at infinity".

- We can define addition between two points.
- Using the "point at infinity" as an identity, this forms a group!
- The j-invariant is a fingerprint: it tells two curves apart up to isomorphism.



Elliptic curves: our vertex building blocks

Definition (Elliptic curve)

Over a field $\mathbb{F}_q$ with charactersitic $q \neq 2, 3$, an *elliptic curve* is the set of (x, y) satisfying $y^2 = x^3 + ax + b$ with $4a^3 + 27b^2 \neq 0$, together with a point at infinity $\mathcal{O}$.

Theorem (Group law)

The points of E form an abelian group under chord-and-tangent addition, with identity $\mathcal{O}$.

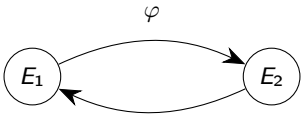
Theorem (Classification [1, III.1.4(b)])

Two elliptic curves are isomorphic over $\overline{\mathbb{F}}_p$ $\iff$ they share the same j-invariant, $j = 1728 \dfrac{4a^3}{4a^3 + 27b^2}$.

For a comprehensive foundation of the math behind elliptic curves, take a look at *The Arithmetic of Elliptic Curves* (Silverman [1])!

Isogenies: our edges

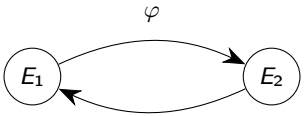
Suppose E_1, E_2 are elliptic curves defined over $\mathbb{F}_q$.
An isogeny $\varphi : E_1 \to E_2$ is a non-constant rational map defined over $\mathbb{F}_q$ that is also a homomorphism.



Isogenies: our edges

Suppose E_1, E_2 are elliptic curves defined over $\mathbb{F}_q$.
An isogeny $\varphi : E_1 \to E_2$ is a non-constant rational map defined over $\mathbb{F}_q$ that is also a homomorphism.

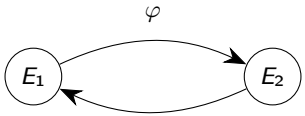
- Preserves the group structure: if $P, Q \in E_1$ then $\varphi(P + Q) = \varphi(P) + \varphi(Q)$



Isogenies: our edges

Suppose E_1, E_2 are elliptic curves defined over $\mathbb{F}_q$.
An isogeny $\varphi : E_1 \to E_2$ is a non-constant rational map defined over $\mathbb{F}_q$ that is also a homomorphism.

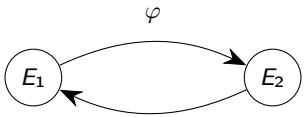
- Preserves the group structure: if $P, Q \in E_1$ then $\varphi(P + Q) = \varphi(P) + \varphi(Q)$
- "Being isogenous" is an **equivalence relation** (there's always a dual isogeny).



Isogenies: our edges

Suppose E_1, E_2 are elliptic curves defined over $\mathbb{F}_q$.
An isogeny $\varphi : E_1 \to E_2$ is a non-constant rational map defined over $\mathbb{F}_q$ that is also a homomorphism.

- Preserves the group structure: if $P, Q \in E_1$ then $\varphi(P + Q) = \varphi(P) + \varphi(Q)$
- "Being isogenous" is an **equivalence relation** (there's always a dual isogeny).
- We say a seperable isogeny has degree ℓ if it has a kernel of size ℓ.



Isogenies: our edges

Definition (Isogeny)

An *isogeny* $\varphi\colon E_1 \to E_2$ is a non-constant morphism defined over $\mathbb{F}_q$ with $\varphi(\mathcal{O}) = \mathcal{O}$. Any such map is automatically a group homomorphism [1, III.4.8].

Definition (Degree [1, II.2])

The *degree* of φ is $\deg \varphi = [\overline{\mathbb{F}}_q(E_1) : \varphi^* \overline{\mathbb{F}}_q(E_2)]$. If φ is separable, then $\deg \varphi = \# \ker \varphi$ [1, III.4.10].

Theorem (Dual isogeny [1, III.6.1])

For every isogeny φ of degree m there is a unique dual $\hat{\varphi}\colon E_2 \to E_1$ with $\hat{\varphi} \circ \varphi = [m]$. In particular, being isogenous is an equivalence relation.

Why *Supersingular*?

Curves come in two types, distinguished by their endomorphism structure ($\mathrm{End}(E)$ is the ring of isogenies of the form $E \to E$):

Why *Supersingular*?

Curves come in two types, distinguished by their endomorphism structure ($\mathrm{End}(E)$ is the ring of isogenies of the form $E \to E$):

- **Ordinary:** commutative symmetry. A quantum computer can take advantage of this extra strcuture (Childs–Jao–Soukharev give a sub-exponential attack [6]).

Why *Supersingular*?

Curves come in two types, distinguished by their endomorphism structure ($\mathrm{End}(E)$ is the ring of isogenies of the form $E \to E$):

- **Ordinary:** commutative symmetry. A quantum computer can take advantage of this extra strcuture (Childs–Jao–Soukharev give a sub-exponential attack [6]).
- **Supersingular:** non-commutative. No known shortcut without this structure.

Why *Supersingular*?

Curves come in two types, distinguished by their endomorphism structure ($\mathrm{End}(E)$ is the ring of isogenies of the form $E \to E$):

- **Ordinary:** commutative symmetry. A quantum computer can take advantage of this extra strcuture (Childs–Jao–Soukharev give a sub-exponential attack [6]).
- **Supersingular:** non-commutative. No known shortcut without this structure.

Also, there are some useful properties of supersingular curves that make things acutally computable. In particular, there are exactly $\ell + 1$ isogenies of degree ℓ (counting multiplicities) originating from any supersingular curve E [5].

Why *Supersingular*?

Definition (Endomorphism Ring [1, III.4])

An *endomorphism* of E is an isogeny $E \to E$, together with the zero map. These form a ring $\mathrm{End}(E)$ under pointwise addition and composition, always containing $\mathbb{Z}$ via the multiplication-by-n maps $[n]$.

Definition (Supersingular Elliptic Curves [1, V.3.1(a)])

We say that an Elliptic curve E is supersingular if $\mathrm{End}(E) \cong \mathcal{O}$ where $\mathcal{O}$ is an order in a quaternion algebra.

Theorem (How many ℓ-isogenies? [5])

Supersingular curves are all defined over $\mathbb{F}_{p^2}$, and for every prime $\ell \neq p$ there are exactly $\ell + 1$ isogenies of degree ℓ (counting multiplicities) originating from any given supersingular curve.

Assemble the Isogeny Graph

- Fix a big prime p, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$

Assemble the Isogeny Graph

- Fix a big prime p, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)

Assemble the Isogeny Graph

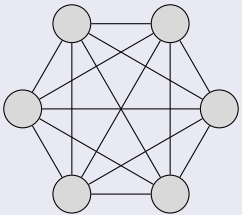
- Fix a big prime p, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ isogenies between two curves

Assemble the Isogeny Graph

- Fix a big prime p, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ isogenies between two curves

Wait... isogenous is an equivalence relation

Isn't this one big, dense blob?



Assemble the ℓ-Isogeny Graph

- Fix a big prime p and a small prime ℓ, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ ℓ-isogenies between two curves

Assemble the ℓ-Isogeny Graph

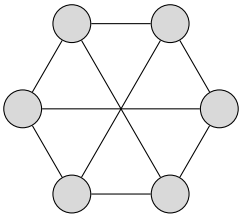
- Fix a big prime p and a small prime ℓ, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ ℓ-isogenies between two curves

Every vertex now has exactly $\ell + 1$ neighbours. The result: an $(\ell + 1)$-regular, connected, undirected graph.

Assemble the ℓ-Isogeny Graph

- Fix a big prime p and a small prime ℓ, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ ℓ-isogenies between two curves

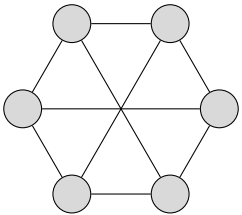
Every vertex now has exactly $\ell + 1$ neighbours. The result: an $(\ell + 1)$-regular, connected, undirected graph.



Assemble the ℓ-Isogeny Graph

- Fix a big prime p and a small prime ℓ, and consider the supersingular elliptic curves over $\mathbb{F}_{p^2}$
- Vertices $\leftarrow$ elliptic curves of an isogeny class (isomorphic curves represent the same vertices)
- Edges $\leftarrow$ ℓ-isogenies between two curves

Every vertex now has exactly $\ell + 1$ neighbours. The result: an $(\ell + 1)$-regular, connected, undirected graph.



The Supersingular Isogeny Path Problem (SIPP)

Given two supersingular curves E_1, E_2 over $\mathbb{F}_{p^2}$, find a path between them in the ℓ-isogeny graph. Equivalently, find an isogeny $E_1 \to E_2$ of degree ℓ^e.

Section 3

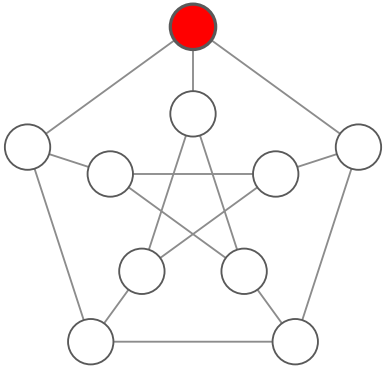
Why is finding a path hard?

Ramanujan graphs: sparse, yet impossibly well-connected

- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.

Ramanujan graphs: sparse, yet impossibly well-connected

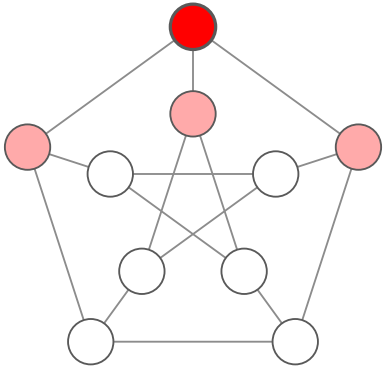
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 0 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.900$

Ramanujan graphs: sparse, yet impossibly well-connected

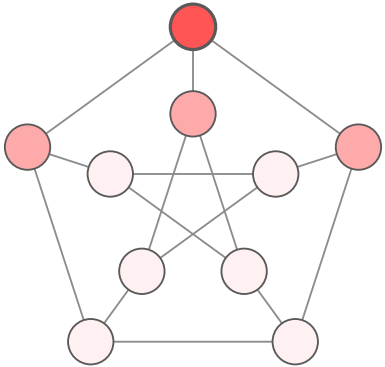
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 1 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.400$

Ramanujan graphs: sparse, yet impossibly well-connected

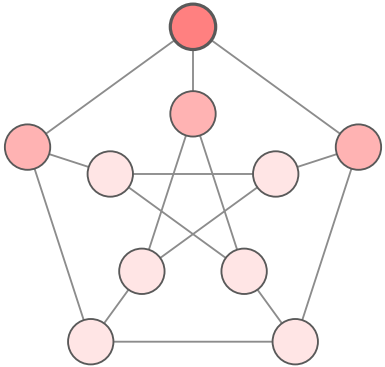
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 2 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.233$

Ramanujan graphs: sparse, yet impossibly well-connected

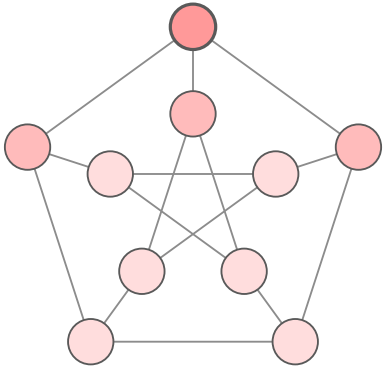
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 3 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.150$

Ramanujan graphs: sparse, yet impossibly well-connected

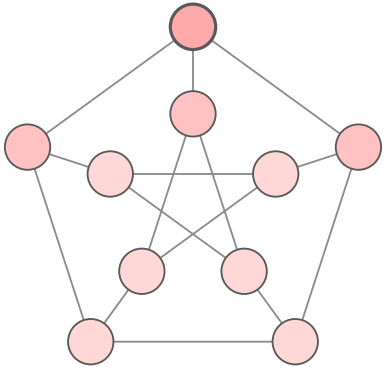
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 4 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.099$

Ramanujan graphs: sparse, yet impossibly well-connected

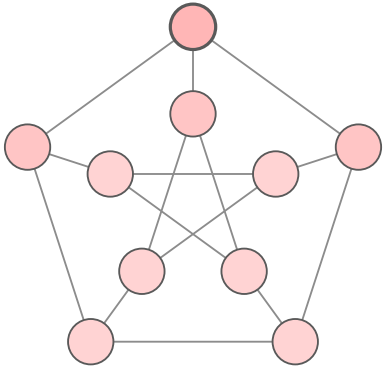
- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 5 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.066$

Ramanujan graphs: sparse, yet impossibly well-connected

- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.



$t = 6 \qquad \max_i \left| p_t(i) - \pi(i) \right| = 0.044$

Ramanujan graphs: sparse, yet impossibly well-connected

- A Ramanujan graph is an *optimal expander*: few edges, yet everything is reachable in very few steps.
- Consequence: a **random walk mixes ridiculously fast**. After a handful of steps you're essentially *uniformly anywhere*.

The key theorem

The supersingular ℓ-isogeny graph **is** a Ramanujan graph [7].

Definition (Ramanujan Graphs [2])

Let $G = (\mathcal{V}, \mathcal{E})$ be a connected, finite k-regular graph on h vertices, with adjacency matrix A, and let $\lambda_{\text{triv}} = k$ denote the trivial eigenvalue afforded by the constant vectors. We say that G is a *Ramanujan graph* if every nontrivial eigenvalue λ of A with $\lambda \neq -k$ satisfies $|\lambda| \leq 2\sqrt{k-1}$.

Theorem (Rapid Mixing [3])

Let G be a regular graph of degree k on h vertices. Suppose that the eigenvalue λ of any nonconstant eigenvector satisfies the bound $|\lambda| \leq c$ for some $c < k$. Let S be any subset of the vertices of G, and x be any vertex in G. Then a random walk of length at least $\frac{\log 2h/|S|^{1/2}}{\log k/c}$ starting from x will land in S with probability at least $\frac{|S|}{2h} = \frac{|S|}{2|G|}$.

Theorem (ℓ-Isogeny Graph is Ramanujan [7])

*The supersingular ℓ-isogeny graph **is** a Ramanujan graph.*

The isogeny backrooms

You wake up in a room. Somewhere out there is *one specific door* you need.



The isogeny backrooms

You wake up in a room. Somewhere out there is *one specific door* you need.

- There are $\sim 2^{256} \approx 10^{77}$ rooms.
  (More than the $\sim 10^{50}$ atoms in the Earth.)



The isogeny backrooms

You wake up in a room. Somewhere out there is *one specific door* you need.

- There are $\sim 2^{256} \approx 10^{77}$ rooms.
  (More than the $\sim 10^{50}$ atoms in the Earth.)
- Each room has only 3 or 4 doors.



The isogeny backrooms

You wake up in a room. Somewhere out there is *one specific door* you need.

- There are $\sim 2^{256} \approx 10^{77}$ rooms.
  (More than the $\sim 10^{50}$ atoms in the Earth.)
- Each room has only 3 or 4 doors.
- The graph mixes so well that running through random doors quickly leaves you uniformly anywhere.



The isogeny backrooms

You wake up in a room. Somewhere out there is *one specific door* you need.

- There are $\sim 2^{256} \approx 10^{77}$ rooms.
  (More than the $\sim 10^{50}$ atoms in the Earth.)
- Each room has only 3 or 4 doors.
- The graph mixes so well that running through random doors quickly leaves you uniformly anywhere.

Without a map, searching *is* just this random wandering.



But what *really* makes it hard?

But what *really* makes it hard?

Honestly? Our own inability.

But what *really* makes it hard?

Honestly? Our own inability.

- **Even with a quantum computer** no fast attack is known. The best attacks stay exponential in the size of p.

But what *really* makes it hard?

Honestly? Our own inability.

- **Even with a quantum computer** no fast attack is known. The best attacks stay exponential in the size of p.
- Its security really rests on the fact that nobody has cracked it *yet*, despite having been investigated extensively by experts.

But what *really* makes it hard?

Honestly? Our own inability.

- **Even with a quantum computer** no fast attack is known. The best attacks stay exponential in the size of p.
- Its security really rests on the fact that nobody has cracked it *yet*, despite having been investigated extensively by experts.

But... maybe you can solve it!

Section 4

From a hard path to a signature

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.
- Each **isogeny** becomes an ideal joining two orders.

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.
- Each **isogeny** becomes an ideal joining two orders.
- The **degree** of an isogeny becomes the norm of its ideal.

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.
- Each **isogeny** becomes an ideal joining two orders.
- The **degree** of an isogeny becomes the norm of its ideal.

In this new interpretation, an algorithm (KLPT [13]) solves our path problem!

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.
- Each **isogeny** becomes an ideal joining two orders.
- The **degree** of an isogeny becomes the norm of its ideal.

In this new interpretation, an algorithm (KLPT [13]) solves our path problem! The catch: computing $\mathrm{End}(E)$ (what you need in order to translate) is itself as hard as finding a path [16].

Our backdoor to the backrooms

The Deuring Correspondence [11] turns our graph into a completely different object: a quaternion algebra. We can reason about these same ideas in this new language:

- Each **curve** becomes a maximal order: its endomorphism ring $\mathrm{End}(E)$.
- Each **isogeny** becomes an ideal joining two orders.
- The **degree** of an isogeny becomes the norm of its ideal.

In this new interpretation, an algorithm (KLPT [13]) solves our path problem! The catch: computing $\mathrm{End}(E)$ (what you need in order to translate) is itself as hard as finding a path [16].

But if you *built* the curve yourself, you already know its endomorphism ring. This is our secret key!

Our backdoor to the backrooms

Theorem (Deuring Correspondence [11])

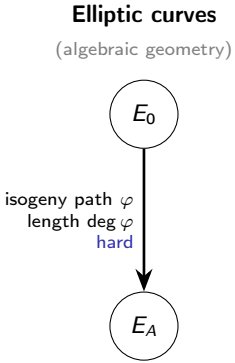
Let $B_{p,\infty}$ be the quaternion algebra over $\mathbb{Q}$ ramified exactly at p and ∞. The map $E \mapsto End(E)$ is a bijection between supersingular curves over $\overline{\mathbb{F}}_p$ (up to Galois conjugacy) and maximal orders in $B_{p,\infty}$ (up to conjugation). Under it, an isogeny $\varphi : E_1 \to E_2$ corresponds to a left $End(E_1)$-ideal I_φ, with $\deg \varphi = nrd(I_\varphi)$ [12, Ch. 42].

Theorem (KLPT, 2014 [13])

Given two maximal orders in $B_{p,\infty}$, there is a heuristically polynomial-time algorithm computing a connecting ideal of ℓ-power norm. This translates to a path in the ℓ-isogeny graph.

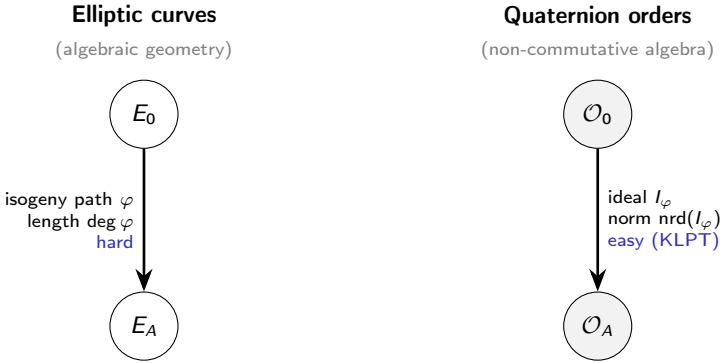
So knowing $\mathrm{End}(E_0)$ and $\mathrm{End}(E_A)$ hands you the path $E_0 \to E_A$. The catch: recovering $\mathrm{End}(E)$ from E alone is itself as hard as finding a path.

Two views of the same problem



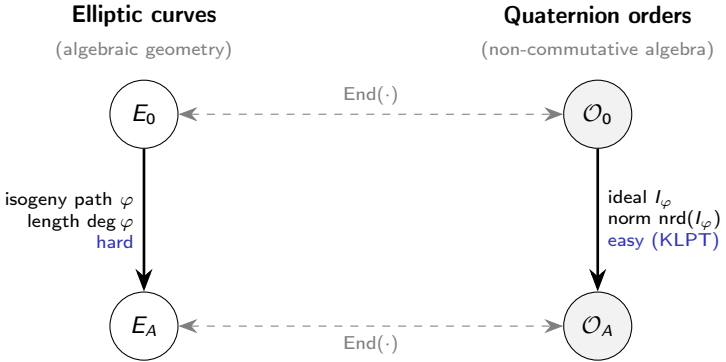
Its the same problem in two languages. Crossing the dashed bridge needs $\mathrm{End}(E)$ (which will be our secret).

Two views of the same problem



Its the same problem in two languages. Crossing the dashed bridge needs $\mathrm{End}(E)$ (which will be our secret).

Two views of the same problem



Its the same problem in two languages. Crossing the dashed bridge needs End(E) (which will be our secret).

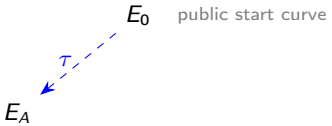
SQISign Step 1: Generate your keys

- Start with a public start curve E_0 where $\mathrm{End}(E_0)$ is known (eg. $y^2 = x^3 + x$, which has $j = 1728$ and is supersingular when $p \equiv 3 \pmod 4$).

E_0 public start curve

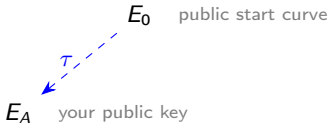
SQISign Step 1: Generate your keys

- Start with a public start curve E_0 where $\mathrm{End}(E_0)$ is known (eg. $y^2 = x^3 + x$, which has $j = 1728$ and is supersingular when $p \equiv 3 \pmod 4$).
- Take a random walk $\tau : E_0 \to E_A$ through the ℓ-isogeny graph.



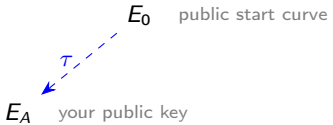
SQISign Step 1: Generate your keys

- Start with a public start curve E_0 where $\mathrm{End}(E_0)$ is known (eg. $y^2 = x^3 + x$, which has $j = 1728$ and is supersingular when $p \equiv 3 \pmod 4$).
- Take a random walk $\tau : E_0 \to E_A$ through the ℓ-isogeny graph.
- Publish E_A as your public key



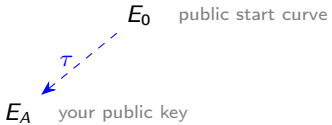
SQISign Step 1: Generate your keys

- Start with a public start curve E_0 where $\mathrm{End}(E_0)$ is known (eg. $y^2 = x^3 + x$, which has $j = 1728$ and is supersingular when $p \equiv 3 \pmod 4$).
- Take a random walk $\tau : E_0 \to E_A$ through the ℓ-isogeny graph.
- Publish E_A as your public key
- Conceal the path τ as your secret key.



SQISign Step 1: Generate your keys

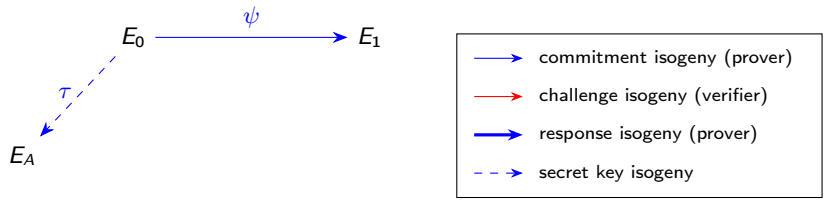
- Start with a public start curve E_0 where $\mathrm{End}(E_0)$ is known (eg. $y^2 = x^3 + x$, which has $j = 1728$ and is supersingular when $p \equiv 3 \pmod 4$).
- Take a random walk $\tau : E_0 \to E_A$ through the ℓ-isogeny graph.
- Publish E_A as your public key
- Conceal the path τ as your secret key.
- Knowing τ also hands you $\mathcal{O}_A = \mathrm{End}(E_A)$.



SQISign Step 2: Come up with an interactive protocol

Prove you know $\mathrm{End}(E_A)$ *without revealing it* [10]:

1. **Commit**: walk a fresh random isogeny $\psi : E_0 \to E_1$, and publish E_1.

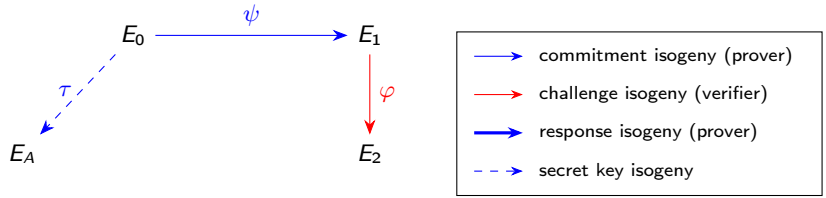


Reproduced from SQISign [10, Fig. 1].

SQISign Step 2: Come up with an interactive protocol

Prove you know $\mathrm{End}(E_A)$ *without revealing it* [10]:

1. **Commit**: walk a fresh random isogeny $\psi : E_0 \to E_1$, and publish E_1.
2. **Challenge**: the verifier picks a random isogeny $\varphi : E_1 \to E_2$.

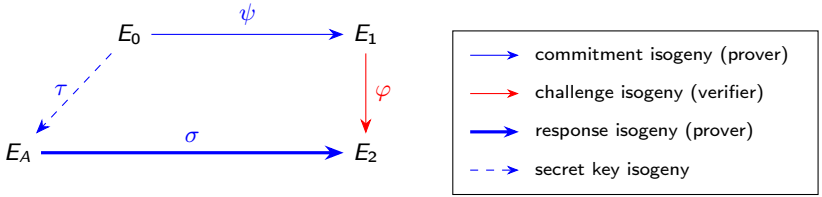


Reproduced from SQISign [10, Fig. 1].

SQISign Step 2: Come up with an interactive protocol

Prove you know $\mathrm{End}(E_A)$ *without revealing it* [10]:

1. **Commit**: walk a fresh random isogeny $\psi : E_0 \to E_1$, and publish E_1.
2. **Challenge**: the verifier picks a random isogeny $\varphi : E_1 \to E_2$.
3. **Response**: using $\mathcal{O}_A = \mathrm{End}(E_A)$, produce an isogeny $\sigma : E_A \to E_2$.

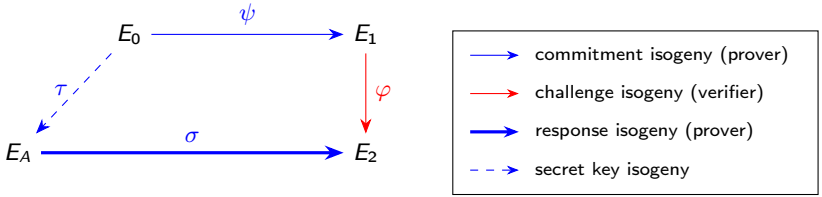


Reproduced from SQISign [10, Fig. 1].

SQISign Step 2: Come up with an interactive protocol

Prove you know $\mathrm{End}(E_A)$ *without revealing it* [10]:

1. **Commit**: walk a fresh random isogeny $\psi : E_0 \to E_1$, and publish E_1.
2. **Challenge**: the verifier picks a random isogeny $\varphi : E_1 \to E_2$.
3. **Response**: using $\mathcal{O}_A = \mathrm{End}(E_A)$, produce an isogeny $\sigma : E_A \to E_2$.
4. **Verify**: check σ really is an isogeny $E_A \to E_2$ of the agreed degree.



Reproduced from SQISign [10, Fig. 1].

SQISign Step 3: Use the Fiat-Shamir transform to make the protocol non-interactive

The verifier's only real job was to provide an unpredictable challenge.

- So replace them with a hash: $\varphi = H(m \,\|\, E_1)$ [8].
- A hash is unpredictable, so you still cannot cook up the challenge in advance.
- Binding the message m into it makes the proof about that message.

Your signature is the pair (E_1, σ). Anyone recomputes $H(m \,\|\, E_1)$ and checks the response. All with no interaction required!

For more on Fiat-Shamir transforms, take CO 485/684!

The lineage of SQISign

- **1941** — Deuring [11]: curves $\leftrightarrow$ quaternion orders.
- **1990** — Pizer [7]: the supersingular ℓ-isogeny graph is Ramanujan.
- **2006** — CGL hash [4]: the isogeny-graph idea enters cryptography.
- **2011** — SIDH, by Jao & De Feo, here at Waterloo [5]: the first supersingular isogeny public-key scheme!
- **2014** — KLPT [13]: the quaternion side of the path problem falls...
- **2020** — SQISign [10]: ...but Deuring + KLPT gives a signature scheme!
- **2022** — SIDH is broken [9]... but because it leaked extra information. THe path problem itself survived.
- **2026** — SQISign [14]: one of nine schemes in NIST's Round 3, and the *only* isogeny one still standing. *(Last minute note: I learned they moved away from using KLPT by now...)*

The idea was pioneered here, and it's still in the race!

Recap

1. Curves are vertices, ℓ-isogenies are edeges.
2. The graph mixes so well that finding a path is hard (Ramanujan $\rightarrow$ backrooms).
3. That hardness builds digital signatures.

And it started right here at Waterloo!

Thank you!

Questions?

References I

[1] J. H. Silverman. *The Arithmetic of Elliptic Curves*. 2nd ed., GTM 106, Springer, 2009.

[2] Alexander Lubotzky, Ralph Phillips, and Peter Sarnak. *Ramanujan graphs*. Combinatorica, 8(3):261-277, 1988.

[3] David Jao, Stephen D. Miller, and Ramarathnam Venkatesan. *Expander graphs based on GRH with an application to elliptic curve cryptography*. J. Number Theory, 129(6):1491-1504, 2009.

[4] D. X. Charles, K. E. Lauter, E. Z. Goren. *Cryptographic hash functions from expander graphs*. Journal of Cryptology, 22(1):93–113, 2009.

[5] D. Jao, L. De Feo. *Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies*. PQCrypto 2011, LNCS 7071, 19–34.

[6] A. Childs, D. Jao, V. Soukharev. *Constructing elliptic curve isogenies in quantum subexponential time*. Journal of Mathematical Cryptology, 8(1):1–29, 2014.

[7] A. K. Pizer. *Ramanujan graphs and Hecke operators*. Bulletin of the AMS, 23(1):127–137, 1990.

References II

[8] A. Fiat, A. Shamir. *How to prove yourself: practical solutions to identification and signature problems*. CRYPTO 1986, LNCS 263, 186–194.

[9] W. Castryck, T. Decru. *An efficient key recovery attack on SIDH*. EUROCRYPT 2023, LNCS 14008, 423–447.

[10] L. De Feo, D. Kohel, A. Leroux, C. Petit, B. Wesolowski. *SQISign: compact post-quantum signatures from quaternions and isogenies*. ASIACRYPT 2020, LNCS 12491, 64–93.

[11] M. Deuring. *Die Typen der Multiplikatorenringe elliptischer Funktionenkörper*. Abh. Math. Sem. Univ. Hamburg, 14:197–272, 1941.

[12] J. Voight. *Quaternion Algebras*. GTM 288, Springer, 2021.

[13] D. Kohel, K. Lauter, C. Petit, J.-P. Tignol. *On the quaternion ℓ-isogeny path problem*. LMS J. Comput. Math., 17(A):418–432, 2014.

[14] G. Alagic et al. *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*. NIST IR 8610, May 2026. doi:10.6028/NIST.IR.8610

References III

[15] K. Eisenträger, S. Hallgren, K. Lauter, T. Morrison, C. Petit. *Supersingular isogeny graphs and endomorphism rings: reductions and solutions*. EUROCRYPT 2018, LNCS 10822, 329–368.

[16] B. Wesolowski. *The supersingular isogeny path and endomorphism ring problems are equivalent*. FOCS 2021, 1100–1111.